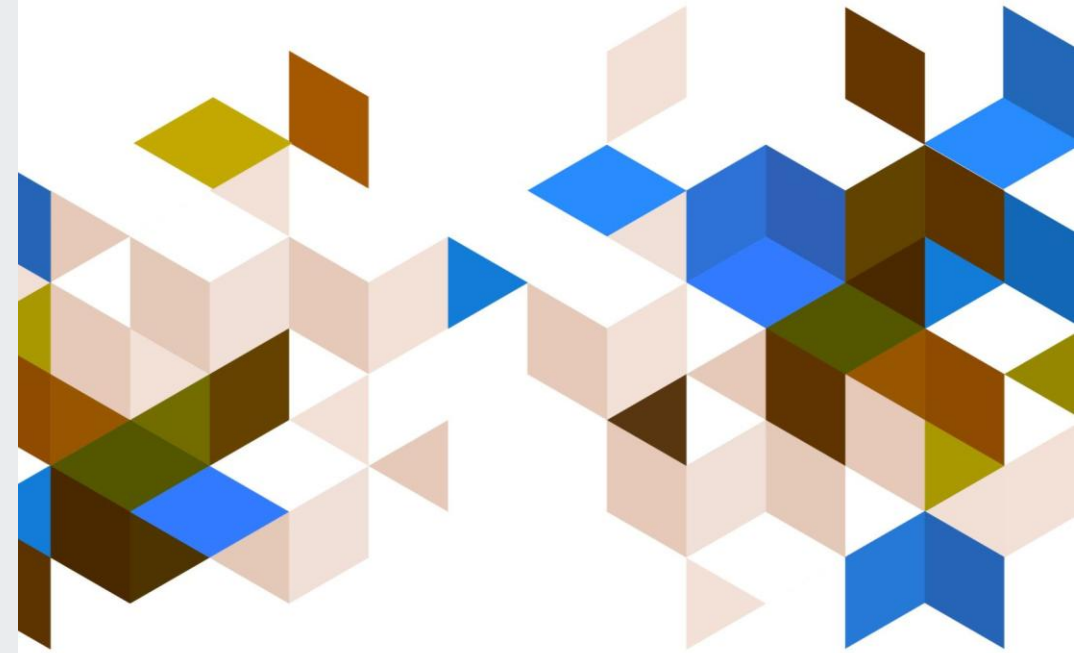


Structures, Unions and Enumerations



Different types of Variables

- int, char, float, double
 - Store value of some element
 - Example: Age, Price, Marks, etc.
- Arrays
 - Collection of values for similar elements
- Pointers
 - Stores addresses
- Representing a real-world object may require capturing values for multiple related elements. For example,
 - Patient name, age, values of medical parameters, vitals
 - Product name, price, seller, features, etc.
- Values of multiple elements together represents the object

Patient Record

Name: John Smith

Date of Birth: March 25, 1965

Sex: Male

Vitals	145/92 mmHg
Blood Pressure:	95 bpm
Respiratory Rate:	20 breaths/min
Body Temperature:	100.2 °F

Diagnosis	Pneumonia
------------------	-----------

Medications

Levofloxacin 750 mg once daily

Acetaminophen 500 mg every 6 hours
as needed

Notes

Start antibiotic treatment. Encourage rest
and fluid intake.

Structures

- Structures are collections of related variables under one name, known as aggregates.
- Structures are derived data types—they're constructed using objects of other types.
 - That is, they contain variables of different types.

Structure Definitions: Example-1

- The keyword *struct* introduces a structure definition
- Example

```
struct employee {  
    char firstName[20];  
    char lastName[20];  
    int age;  
    double hourlySalary;  
};
```

- employee is a structure tag
- Variables declared inside a structure are the structure's members
- Each structure definition ends with a semicolon

Structure Definitions: Example-2

- Example

```
struct card {  
    char *face;  
    char *suit;  
};
```

- Here, again, `struct` introduces the definition for structure `card`
- `card` is the structure name/tag and is used to declare variables of the structure type -- '`struct card`'
- `card` contains two members of type `char *`
 - These members are `face` and `suit`

More on Structure Definitions

- struct information
 - A struct cannot contain an instance of itself (i.e., a variable of its own struct type)
 - But can contain a member that is a pointer to the same structure type (Self-referential structure)

```
struct employee {  
    char firstName[20];  
    char lastName[20];  
    int age;  
    double hourlySalary;  
    struct employee *managerPtr;  
};
```

Defining Variables of Structure Type

- A structure definition does not reserve space in memory
 - Instead creates a new data type used to define structure variables
 - Structures are user defined data types
 - Variables of structure types can be created
- Defining Variables: Following statements reserve memory for variables using type 'struct card'
 - Defined like other variables:

```
struct card oneCard, deck[ 52 ], *cPtr;
```
 - Can use a comma separated list (variables can also be created while defining the type):

```
struct card {  
    char *face;  
    char *suit;  
} oneCard, deck[ 52 ], *cPtr;
```

Operations Allowed on Structures

- Assigning one struct variable to another of the same type
 - `firstEmployee = newEmployee;`
- Taking the address (&) of a struct variable
 - `struct employee *ePtr = &firstEmployee;`
- Accessing a struct variable's members
- Zero initializing a struct variable in its definition
 - `struct employee emp = {};`
- Using the `sizeof` operator to determine a struct variable's size
 - `sizeof(employee)`

Operations NOT Allowed on Structures

- Assigning a structure of one type to one of a different type: compilation error.
- Direct comparison using `==` and `!=` operators are not allowed
- Consider the following
struct example {
 char c;
 int i;
} sample1, sample2;
- Each struct member might be required to be aligned on a word boundary.
- A **word** is a standard memory unit used to store data in a computer, usually 2 bytes or 4 bytes.

Byte offset	Content	Explanation
0	c	1 byte for the <code>char</code>
1–3	padding	3 unused bytes to align <code>i</code> on 4-byte boundary
4–7	i	4 bytes for the <code>int</code>

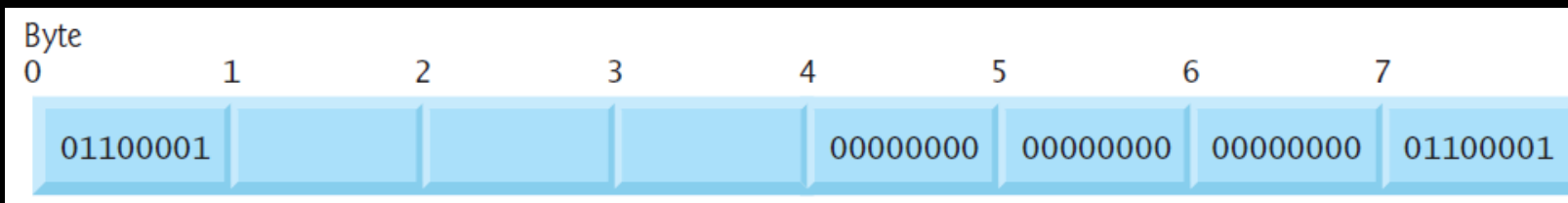
Direct comparison of Structures

- Structure members are not necessarily stored in consecutive bytes of memory. Sometimes there are “holes” in a structure, because computers may store specific data types only on certain memory boundaries such as **half word**, **word** or **double word boundaries**. The following example assumes four-byte word.

e.g.

```
struct example {  
    char c;  
    int i;  
} sample1, sample2;
```

Even if the member values of sample1 and sample2 are equal, the structures are not necessarily equal, because the undefined 1-byte holes are not likely to contain identical values.



Initializing Structures

- Initializer lists
 - Example:

```
struct card oneCard = { "Three", "Hearts" };
```
 - If there are fewer initializers in the list than members in the structure,
 - the remaining members are automatically initialized to 0 (or NULL if the member is a pointer).
- Assignment statements
 - Example:

```
struct card threeHearts = oneCard;
```
 - Could also define and initialize threeHearts as follows:

```
structure card threeHearts;  
threeHearts.face = "Three";  
threeHearts.suit = "Hearts";
```

Accessing Members of Structures

- Accessing structure members
 - Dot operator (.) used with structure variables

```
struct card myCard;  
printf( "%s", myCard.suit );
```
 - Arrow operator (->) used with pointers to structure variables

```
struct card *myCardPtr = &myCard;  
printf( "%s", myCardPtr->suit );
```
 - `myCardPtr->suit` is equivalent to
`( *myCardPtr ).suit`
- Dot operator and arrow operator: highest operator precedence

Using Structures: Example

```
1  /* Fig. 10.2: fig10_02.c
2     Using the structure member and
3     structure pointer operators */
4  #include <stdio.h>
```

```
5
```

```
6  /* card structure definition */
```

```
7  struct card {
```

Structure definition

```
8     char *face; /* define pointer face */
```

```
9     char *suit; /* define pointer suit */
```

```
10 }; /* end structure card */
```

Structure definition must end with semicolon

```
11
12 int main( void )
```

```
13 {
```

```
14     struct card aCard; /* define one struct card variable */
```

```
15     struct card *cardPtr; /* define a pointer to a struct card */
```

```
16
```

```
17     /* place strings into aCard */
```

```
18     aCard.face = "Ace";
```

```
19     aCard.suit = "Spades";
```

Dot operator accesses members of a structure

Using Structures: Example

```
20
21  cardPtr = &aCard; /* assign address of aCard to cardPtr */
22
23  printf( "%s%s\n%s%s\n%s%s\n", aCard.face, " of ", aCard.suit,
24          cardPtr->face, " of ", cardPtr->suit,
25          ( *cardPtr ).face, " of ", ( *cardPtr ).suit );
26
27  return 0; /* indicates successful termination */
28
29 } /* end main */
```

Ace of Spades
Ace of Spades
Ace of Spades

Arrow operator accesses members
of a structure pointer

Using Structures with Functions

- Passing structures to functions
 - Pass entire structure
 - Or, pass individual members
 - Both pass call by value
- To pass structures call-by-reference
 - Pass its address
- To pass arrays call-by-value
 - Create a structure with the array as a member
 - Pass the structure

typedef

- typedef
 - Creates synonyms (aliases) for previously defined data types
 - Use typedef to create shorter type names
 - Example:

```
typedef struct Card *CardPtr;
```
 - Defines a new type name CardPtr as a synonym for type struct Card *
 - typedef does not create a new data type
 - Only creates an alias of an existing type name

Unions

- A union is a derived data type with members that share the storage space.
- **Use-case:** a communication system where a packet can contain either sensor data or a text message.
- An event queue that can hold *any* event efficiently (mouse, keyboard, resize), where only one is valid per event.
- **union**
 - Members share the same storage space
 - The members of a union can be of any data type
 - The number of bytes used to store a union must be at least enough to hold the largest member
 - Only one member, and thus one data type, can be referenced at a time
- **union definitions**
 - Same as struct, e.g.

```
union Number {  
    int x;  
    float y;  
};  
union Number value;
```

Unions

- Valid union operations
 - Assignment to union of same type: =
 - Taking address: &
 - Accessing union members: .
 - Accessing members using pointers: ->
- In a declaration, a union may be initialized with a value of the same type as the first union member, e.g.
`union number value = {10};`
- `union number value = {1.43}`
would truncate the floating-point part of the initializer value and normally would produce a warning from the compiler.

Unions: Example

```
1  /* Fig. 10.5: fig10_05.c
2     An example of a union */
3  #include <stdio.h>
4
5  /* number union definition */
6  union number {
7     int x;
8     double y;
9 }; /* end union number */
10
11 int main( void )
12 {
13     union number value; /* define union variable */
14
15     value.x = 100; /* put an integer into the union */
16     printf( "%s\n%s\n%s\n %d\n\n%s\n %f\n\n\n",
17         "Put a value in the integer member",
18         "and print both members.",
19         "int:", value.x,
20         "double:", value.y );
21
```

Union definition

Union definition must end with semicolon

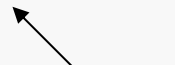
Note that **y** has no value

Unions: Example

```

22 value.y = 100.0; /* put a double into the same union */
23 printf( "%s\n%s\n%s\n  %d\n\n%s\n  %f\n",
24         "Put a value in the floating member",
25         "and print both members.",
26         "int:", value.x,
27         "double:", value.y );
28
29 return 0; /* indicates successful termination */
30
31 } /* end main */

```



Given

Giving **y** a value removes **x**'s value

Put a value in the integer member and print both members.

```
int:
    100
```

double:

[illegible]

Put a value in the floating member and print both members.

```
int:
    0
```

double:

100.000000

Enumeration Constants

- Enumeration
 - Set of integer constants represented by identifiers
 - Improves **code readability** over plain integers.
 - Example:

```
enum Months { JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC};
```

 - Creates a new type enum Months in which the identifiers are set to the integers 1 to 12
 - Enumeration constants are like symbolic constants whose values are automatically set
 - Values start at 0 and are incremented by 1
 - Values can be set explicitly with =
 - Need unique constant names
 - Enumeration variables can only assume their enumeration constant values (not the integer representations)

```
enum ErrorCode { SUCCESS = 0, WARNING = 1, FATAL = 100 };
```

Enums: Example

```
1  /* Fig. 10.18: fig10_18.c
2     Using an enumeration type */
3  #include <stdio.h>
4
5  /* enumeration constants represent months of the year */
6  enum months {
7     JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC };
8
9  int main( void )
10 {
11     enum months month; /* can contain any of the 12 months */
12
13     /* initialize array of pointers */
14     const char *monthName[] = { "", "January", "February", "March",
15         "April", "May", "June", "July", "August", "September", "October",
16         "November", "December" };
17
```



Enumeration sets the value of constant **JAN** to 1 and the following constants to 2, 3, 4...

Enums: Example

```
18  /* loop through months */
19  for ( month = JAN; month <= DEC; month++ ) {
20      printf( "%2d%11s\n", month, monthName[ month ] );
21  } /* end for */
22
23  return 0; /* indicates successful termination */
24 } /* end main */
```

```
1  January
2  February
3  March
4  April
5  May
6  June
7  July
8  August
9  September
10 October
11 November
12 December
```

Like symbolic constants, enumeration constants are replaced by their values at compile time